

Report - Version 5: Memory Manager

Introduction

Version 5 introduces the Memory Manager.

In the previous version, the project introduced the Planner. The agent was already able to select a tool, create an execution plan, extract the required arguments, execute the selected tool, and store the interaction.

Version 5 keeps this structure, but adds a new way to manage memory: the Agent no longer writes directly to the memory list.

This version is still rule-based. No LLM is used, no database is introduced, and no external framework is introduced.

Goal of Version 5

The main goal of Version 5 is to introduce cleaner memory management.

The Agent should no longer be directly responsible for saving, reading, or clearing memory. It should delegate these operations to a dedicated component.

The Memory Manager introduces simple functions to manage memory:

- `save_to_memory(state);`
- `get_memory();`
- `get_last_interaction();`
- `clear_memory();`

The important idea is simple: memory should not only exist, it should be managed.

What changes compared to Version 4

Version 4 focused on the Planner. The Planner introduced a plan before parsing and execution.

Version 5 does not modify the Planner as the main concept. The Planner continues to create a plan with the selected tool, the reason for using it, and the required arguments.

The new component is the Memory Manager.

The flow changes from:

Request → Router → Planner → Generic Parser → Executor → Memory

to:

Request → Router → Planner → Generic Parser → Executor → Memory Manager → Memory

The Executor continues to produce the result. The Memory Manager then handles saving, reading, or clearing the interaction state.

New concept: Memory Manager

The Memory Manager is a new component that manages memory operations.

In this version, memory remains a simple list. The difference is that it is no longer used directly by the Agent.

If a state needs to be saved, the Agent no longer calls directly:

```
memory.append(state)
```

but delegates to the Memory Manager:

```
save_to_memory(state)
```

This makes the role of the Agent cleaner. The Agent coordinates the flow, while the Memory Manager manages memory.

Memory Manager functions

Version 5 introduces simple and readable functions.

The function `save_to_memory(state)` saves the current state to memory.

The function `get_memory()` returns the full memory.

The function `get_last_interaction()` returns the last saved interaction. If memory is empty, it returns `None`.

The function `clear_memory()` clears memory.

These functions make memory easier to control during tests and prepare the project for more advanced memory management in future versions.

Complete architecture in Version 5

The complete flow of Version 5 is:

Request → Router → Planner → Generic Parser → Executor → Memory Manager → Memory

↓

Output

The main responsibilities are now clearer:

Router = selects the tool

Planner = creates the execution plan

Parser = extracts arguments

Executor = validates and executes

Memory Manager = saves, reads, and clears memory

Agent = coordinates

Memory = stores the state

The Agent is still the coordinator. It now coordinates planning, parsing, execution, and memory storage through the Memory Manager.

Agent state in Version 5

The agent state keeps the same structure introduced in Version 4.

A typical state looks like this:

```
{
  "request": "What is 2 + 3?",
  "plan": {
    "tool": "addition",
```

```

    "reason": "Add two numbers.",
    "needs_arguments": ["a", "b"]
  },
  "action": "addition",
  "arguments": {"a": 2, "b": 3},
  "result": 5
}

```

The difference is that this state is saved through the Memory Manager, not through a direct write to the memory list.

Unsupported requests

Unsupported requests are also handled correctly.

For example, the request:

```
agent("What is the weather in Rome?")
```

does not match any available tool.

The Planner returns a plan with no selected tool and the Agent returns a safe fallback result.

The new point in Version 5 is that this interaction is also saved through the Memory Manager.

This is important because memory should record both supported and unsupported requests.

Tests

The tests verify that the agent continues to work after adding the Memory Manager.

The main test cases are:

```

clear_memory()
agent("What is 2 + 3?")
agent("Hello, what is 2 + 3?")
agent("Hello, my name is luca.")
agent("What is the weather in Rome?")
get_memory()
get_last_interaction()

```

These tests verify that:

- the Router still prioritizes the main intent;
- the Planner still creates a plan before execution;
- the Parser still extracts the correct arguments;
- the Executor still validates and executes the selected tool;
- unsupported requests are handled without selecting a tool;
- the Memory Manager saves, reads, and clears memory;
- memory keeps the full state of the interactions.

What Version 5 teaches

Version 5 teaches an important agent design principle:

memory should be managed by a dedicated component

The Memory Manager is still simple, but it introduces an important separation between coordination and state management.

This makes the architecture easier to extend in the future, especially when memory becomes more complex or when the project moves toward a more professional structure.

Limitations

Version 5 improves the architecture, but some limitations remain:

- the Router is still rule-based;
- the Planner is still rule-based;
- the Parser still uses manual extraction rules;
- memory is still a simple list;
- the Memory Manager does not use a database or external storage yet;
- there is no Tool Registry yet;
- there is no LLM yet;
- the agent still performs only one action at a time.

These limitations are expected and will be addressed gradually in the next versions.

Conclusion

Version 5 is a natural evolution of Version 4.

The Planner remains unchanged, but the new central element is the Memory Manager.

The Agent no longer writes directly to memory. It now delegates saving, reading, and clearing memory to dedicated functions.

This makes the agent cleaner, more organized, and closer to a professional structure.

The next step will be Version 6, where the project will introduce a Tool Registry.